

Document 3.pdf

 Shri Vile Parle Kelavani Mandal

Document Details

Submission ID

trn:oid::9832:137046033

Submission Date

Apr 29, 2026, 2:31 PM GMT+5:30

Download Date

Apr 29, 2026, 2:37 PM GMT+5:30

File Name

Document 3.pdf

File Size

302.9 KB

28 Pages

6,775 Words

38,218 Characters

74% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups



70 AI-generated only 74%

Likely AI-generated text from a large-language model.



0 AI-generated text that was AI-paraphrased 0%

Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



Locus — A Gamified Learning Platform for Indian Competitive Examinations

Capstone Project Report

Submitted by: Vinay Tiwari **Project name:** Locus **Repository:** github.com/vinaytz/Locus **Submission date:** April 2026

Declaration

I declare that the work presented in this report is the result of my own effort. The Locus platform — including its database schema, backend services, mobile application and administrative dashboard — was designed and implemented by me. External libraries, frameworks and infrastructure services have been credited in the references. Where ideas were inspired by existing products (notably Duolingo, whose game mechanics influenced the gamification layer), this has been stated explicitly inside the relevant section. No part of this report has been submitted for any other academic award.

Table of Contents

1. Abstract
2. Introduction
3. Problem Statement and Motivation
4. Objectives and Scope
5. Literature and Product Survey
6. System Architecture
7. Database Design
8. Backend Design and Implementation
9. Gamification Subsystem
10. Leaderboard Subsystem
11. Mobile Application
12. Administrative Dashboard

13. Content Authoring and Seed Pipeline
14. Deployment and DevOps
15. Security Considerations
16. Testing and Validation
17. Results
18. Limitations
19. Future Work
20. Conclusion
21. References
22. Appendix A — Sample API Requests
23. Appendix B — Question Content Schemas
24. Appendix C — File and Module Inventory

1. Abstract

Locus is a full-stack mobile learning platform built around the syllabi of five Indian competitive examinations: JEE Main, NEET, NIMCET, CUET and UPSC CSE. The application borrows the lesson-and-streak loop popularised by Duolingo and reapplies it to question-based subject preparation. A learner picks an exam (or, alternatively, a single subject and a difficulty band), receives a feed of bite-sized exercises, and is rewarded with experience points, gem-style "diamonds", a daily streak counter and weekly placement in a thirty-tier league. The system was implemented across four sub-projects: a NestJS backend exposing a versioned REST API, a PostgreSQL data store managed through Prisma, a React Native client built on Expo, and a Next.js administrative dashboard for content authoring. The complete application has been packaged with Docker, deployed to a DigitalOcean droplet alongside an Upstash Redis instance, and a signed Android App Bundle has been submitted to the Google Play internal track. At the time of writing the database carries 24 distinct subject records, 124 units, 465 exercises and 4,650 questions, all generated from a deterministic seed pipeline that lives inside the repository. This report documents the requirements, design, implementation, deployment and measured behaviour of the platform.

2. Introduction

Indian competitive examinations follow a small number of well-defined syllabi, but the resources around them are fragmented. Aspirants typically juggle physical books, PDF question banks, YouTube playlists, scattered Telegram groups and a handful of paid

web portals that are heavy, slow on poor connections, and not designed to be opened in short bursts. There is a behavioural gap as well: most existing tools treat preparation as a long-form session — an hour of mock tests, a lecture, a chapter — and not as something that can be done in three-minute pockets while waiting for a bus.

Duolingo solved a structurally similar problem in language learning by reducing each session to a handful of small interactions and by wrapping those interactions in a clear feedback loop: lose a heart on a wrong answer, gain experience on a right one, build a streak by returning daily, and compete with eleven anonymous strangers on a weekly leaderboard. The mechanics are well-studied and have been credited with substantial increases in retention.

Locus is an attempt to reapply this loop to entrance-exam preparation. The pedagogical content — questions, options, explanations — is deliberately kept small per session: a single exercise contains roughly ten questions and is meant to be completed in five minutes. The motivational scaffolding around it (XP, hearts, diamonds, streaks, leagues) is implemented in full. The result is a mobile-first product that is both pedagogically focused and behaviourally engaging.

This report describes how the platform was designed and built, from the database schema up to the production deployment.

3. Problem Statement and Motivation

The problem this project addresses can be stated as follows. Given a set of standardised exam syllabi and a population of learners with limited daily attention, design and implement a software system that (a) presents the syllabus content as small, consumable interactions, (b) measures learner progress and rewards consistent practice, (c) offers light social pressure through ranked competition, and (d) can be operated and updated by a small team using inexpensive infrastructure.

The motivations were threefold. Academically, the project provided an opportunity to integrate a number of areas covered during the degree — relational database design, REST API engineering, mobile development, real-time data structures, containerised deployment and authentication — into a single coherent system. Practically, I had no existing answer to the question "what should I use to prepare for X exam in five-minute pockets?", and wanted one that was not advertisement-laden. Strategically, the project was scoped so that a working version could be shipped end-to-end within a capstone timeline and continue to be useful afterwards.

4. Objectives and Scope

4.1 Primary objectives

The platform was required to satisfy the following functional objectives.

- Allow a learner to register, authenticate, choose either an exam plus an exam date or a subject plus a difficulty level, and persist that choice.
- Serve the syllabus as a four-level hierarchy: Subject → Unit → Exercise → Question.
- Support five distinct question types: multiple choice, matching, reordering, fill-in-the-blank and translation.
- Track and award experience points for each exercise completed, with a bonus for a perfect score.
- Deduct one heart on every wrong answer and refill hearts either by waiting (one heart every thirty minutes) or by spending diamonds.
- Track a daily streak in the user's local timezone, with a milestone reward every seventh day.
- Place every active user into a twelve-person league group, run a ten-day season per group, and apply promotion or demotion at the end of the season based on relative experience earned.
- Provide an administrator-only dashboard for creating and editing all content.
- Deploy the backend so that the mobile client can talk to it from outside the development laptop.

4.2 Non-objectives

To keep the scope realistic the following were explicitly out of scope.

- Real-money payments. The diamond economy is closed; diamonds are earned, not bought.
- Live tutor-led classes or video content.
- iOS submission. The build targets Android only for the capstone milestone.
- Multi-language content. The seed corpus is in English.

5. Literature and Product Survey

A short survey of comparable products informed the design.

Duolingo was the primary influence on the gamification loop: the streak counter, the weekly league, the heart-based limit on incorrect answers, and the use of a single bottom-of-screen "continue" button were all observed there before being adapted. The leaderboard sizing of twelve participants and the ten-day cadence were taken from the publicly described behaviour of the Duolingo Diamond league prior to its 2024 redesign.

Brilliant and **Khan Academy** influenced the way exercises are framed as small, self-contained learning units that immediately give feedback. Unlike either of those products, Locus does not attempt to teach a topic from scratch — it assumes the learner already has source material and is using the app for spaced practice.

Unacademy and **Physics Wallah** were studied as examples of the existing Indian market. Their products are video-heavy and oriented around live and recorded classes; the gap that Locus targets — short, asynchronous, gamified question practice — is not their core offering.

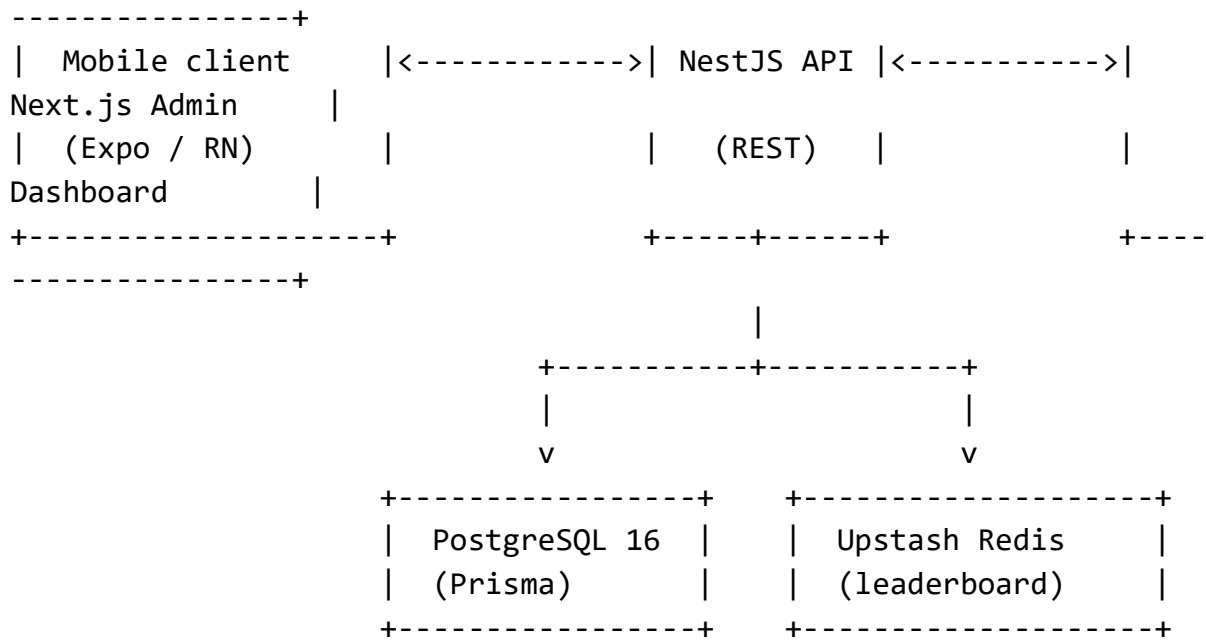
The database design borrows from a well-known pattern in adaptive-learning platforms (referred to internally as the "scalable MCQ schema") in which the question payload is stored as a flexible JSON document while the surrounding hierarchy is fully relational. This combination keeps query patterns simple — every list view is a join over indexed foreign keys — while avoiding the proliferation of one table per question type.

6. System Architecture

6.1 Overall shape

The system is composed of four sub-projects that communicate over HTTP and share a single PostgreSQL database.





The four sub-projects are kept under one Git repository to simplify deployment and to keep the schema, server and client types in sync.

6.2 Sub-project responsibilities

Sub-project	Folder	Role
Backend	backend/	REST API, business logic, data access, background jobs
Mobile	mobile/	End-user Android application
Admin	admin/	Content-author web dashboard
Course data	Courses/	JSON corpus and helper scripts that produced the seed

The boundary between backend and the two clients is the REST contract documented in [Appendix A](#). The boundary between backend and the Postgres instance is the Prisma schema described in section 7.

6.3 Technology choices

Layer	Technology	Version	Reason for choice
-------	------------	---------	-------------------

API framework	NestJS	10.3.10	Module system maps cleanly to feature folders; first-class dependency injection.
ORM	Prisma	5.18.0	Strongly-typed client, declarative migrations, good developer experience.
Relational store	PostgreSQL	16	Mature, indexed, JSONB-capable, free.
Cache and ranking	Redis (Upstash)	7	$O(\log N)$ sorted-set operations are exactly what the leaderboard needs.
Auth	JSON Web Tokens (Passport JWT)	10.2.0	Stateless, simple to verify on the mobile client.
Password hashing	bcryptjs	2.4.3	Adaptive cost factor; widely audited.
Mobile platform	Expo + React Native	51.0.28 / 0.74.5	Single codebase, OTA-friendly, EAS build pipeline for Play Store.
Mobile router	expo-router	3.5.23	File-system routing, mirrors Next.js conventions.
Admin dashboard	Next.js	14.2.15	Server-rendered React with built-in routing.
Containerisation	Docker + Compose	29.x	Reproducible deploys across machines.

7. Database Design

7.1 Conceptual model

Conceptually the data falls into four clusters.

1. **Identity** — the User record and its hashed credentials.

2. **Content** — Subject, Unit, Exercise, Question, plus the Exam aggregate that groups subjects.
3. **Onboarding** — UserExamEnrollment and UserSubjectEnrollment, which capture what a learner has chosen to study.
4. **Activity** — UserStats, ExerciseAttempt, DiamondLedger and the leaderboard pair (LeaderboardGroup, LeaderboardParticipant), which capture what the learner has done and their position relative to peers.

7.2 The content hierarchy

Each Subject row has a level of 1, 2 or 3, which corresponds in the seed to roughly Class 10, Class 12 and graduate-level material. The same name ("Physics") therefore appears three times with distinct level values. The composite uniqueness constraint (name, level) on the table prevents duplicates.

A Unit belongs to a Subject. An Exercise belongs to a Unit and carries a base point value (default 20 XP) and a duration in minutes (default 5). A Question belongs to an Exercise, has a type from the set {MCQ, MATCH, REORDER, COMPLETE, TRANSLATE}, a difficulty tag, a points value, and a content field carrying the type-specific payload as a JSON string.

The decision to store content as String (TEXT) rather than as Postgres JSONB was made deliberately. A string column behaves identically across SQLite (used during early prototyping) and PostgreSQL, which removed an entire class of migration risk when the database backend was switched. The application code already needs to validate the payload in any case, so the loss of native JSON operators inside the database is not material.

7.3 Exam aggregation

Exam and Subject have a many-to-many relationship via ExamSubject. The same Subject row can appear in multiple exams without being duplicated; for instance "Mathematics 2" is shared by JEE Main, NIMCET, CUET and UPSC. The join table carries an orderIndex so the admin can control the order of subjects within an exam.

7.4 Activity records

Every completed exercise produces an ExerciseAttempt row that stores correctCount, totalCount, the integer percentage, the XP awarded, the diamonds awarded, and a isPerfect boolean. This table is the permanent ledger of learner activity and is the basis on which "completed exercise" sets are derived for the home

screen. Diamond changes are independently logged to DiamondLedger with a reason string, which makes it straightforward to audit the in-app currency.

7.5 Schema summary

The complete model count is fifteen. A condensed view of the most important fields appears below; the full source is in [backend/prisma/schema.prisma](#).

Model	Key fields
User	id, email (unique), passwordHash, displayName, timezone
Subject	id, slug (unique), name, level, displayName, (name, level) unique
Unit	id, subjectId, title, orderIndex
Exercise	id, unitId, title, points, duration
Question	id, exerciseId, type, prompt, difficulty, points, content (JSON string)
Exam	id, slug, title
ExamSubject	(examId, subjectId) composite key, orderIndex
UserExamEnrollment	userId, examId, examDate
UserSubjectEnrollment	userId, subjectId, difficulty (1 2 3)
UserStats	userId (PK), totalXp, diamonds, hearts, lastHeartRefillAt, currentLeagueIndex, currentStreak, longestStreak, lastActivityDate
ExerciseAttempt	id, userId, exerciseId, correctCount, totalCount, scorePct, xpAwarded, diamondsAwarded, isPerfect
DiamondLedger	id, userId, delta, reason, refId
LeaderboardGroup	id, leagueTier, state, capacity (12), filledCount, startedAt, endsAt

LeaderboardParticipant	groupId, userId, seasonXp, joinedAt, lastXpAt
------------------------	---

7.6 Indexing

Indexes were placed on every foreign key that drives a list query. Notable ones include (userId, createdAt) on ExerciseAttempt (used to compute the timeline view), (leagueTier, state) on LeaderboardGroup (used to find an open group quickly), and the partial index on (groupId, userId) on LeaderboardParticipant (used to enforce one membership per group). All indexes are declared in the Prisma schema and are propagated automatically through `prisma migrate`.

8. Backend Design and Implementation

8.1 Module layout

The backend is organised as ten NestJS modules. Each module owns one feature, declares its providers explicitly, and exposes a controller class that maps HTTP routes to service methods. The module list is reproduced from [backend/src/app.module.ts](#).

Module	Responsibility
ConfigModule	Reads <code>.env</code> and exposes a typed <code>ConfigService</code> .
ScheduleModule	Registers cron-style background jobs.
PrismaModule	Owens a single global <code>PrismaClient</code> .
RedisModule	Owens a single global <code>ioredis</code> client; degrades gracefully when <code>REDIS_URL</code> is absent.
AuthModule	Registration, login, JWT issuance, JWT verification.
UsersModule	The current user, their enrollments, their stats.
CoursesModule	Read-only views of the content hierarchy.
AdminModule	CRUD over every content table; protected by an admin token.
GamificationModule	Exercise completion and the rewards that flow from it.

LeaderboardModule	League state, group membership, ranked reads.
JobsModule	The daily streak-reset worker.

8.2 Bootstrap

The application is bootstrapped in `backend/src/main.ts`. Three things are configured before the server starts listening: the global `/api` prefix, a `ValidationPipe` with `whitelist: true` and `transform: true` so that incoming DTOs are stripped of unknown fields and coerced to the right types, and CORS, which is enabled for the mobile and admin clients.

8.3 Authentication

`AuthService` exposes two methods: `register` and `login`. Both produce the same shape — a `User` summary plus a signed JWT — through a private `signAndShape` helper. Passwords are hashed with a `bcrypt` cost factor of 10. The JWT payload carries only the `sub` (user id) and `email` claims; the secret and the expiry window are read from `JWT_SECRET` and `JWT_EXPIRES_IN`.

Verification is delegated to `JwtStrategy`, which extracts the bearer token from the `Authorization` header and resolves the `User` from the database. A helper decorator, `@CurrentUser()`, makes the resolved user available to controllers without repetitive plumbing. Routes that require authentication declare `@UseGuards(JwtAuthGuard)`.

8.4 The courses module

`CoursesController` exposes five endpoints, all read-only.

Method	Path	Purpose
GET	<code>/api/courses/exams</code>	List all exams with their subjects.
GET	<code>/api/courses/subjects?level=1 2 3</code>	List subjects, optionally filtered by level.
GET	<code>/api/courses/subjects/:id</code>	A subject with its units and exercises.

GET	/api/courses/units/:id	A unit with its exercises and parent subject.
GET	/api/courses/exercises/:id	An exercise with its full question list.

The questions returned from `/exercises/:id` are run through a small parser that calls `JSON.parse` on the stored content string and returns a structured object so the mobile client never has to do the parse itself.

8.5 The admin module

The admin module mirrors the content hierarchy with full CRUD endpoints — subjects, exams, units, exercises and questions all support GET (paginated), POST, PUT and DELETE. Every admin route is protected by `AdminGuard`, which compares the `x-admin-token` header against the `ADMIN_API_TOKEN` environment variable. This keeps the admin surface separate from the user surface and avoids the need to give administrators full JWT-issued accounts.

The admin POST `/questions` and PUT `/questions/:id` endpoints accept the question payload as an object and serialise it with `JSON.stringify` before storing. The mirrored read endpoint deserialises it again. This keeps the API ergonomic on both sides without sacrificing the storage decision documented in section 7.

9. Gamification Subsystem

The gamification module is the most behaviourally significant part of the system. Every meaningful state change happens through `GamificationService.completeExercise`, which is the only place that writes to `UserStats` and `DiamondLedger`. Centralising the writes keeps the state machine traceable.

9.1 The completion path

When a user finishes an exercise the mobile client posts an array of booleans, one per question, to POST `/api/exercise/complete`. The service then performs the following sequence inside a Prisma transaction.

1. Load the exercise and its questions; refuse the request if the count of booleans does not match.
2. Compute `correctCount` and `totalCount`.
3. Apply heart loss: a `loseHearts` call is made for each incorrect answer, capped so the user is never charged more hearts than they currently have. If the user has zero hearts the entire request is rejected with HTTP 403.
4. Compute XP: a base of 20 plus a perfect-score bonus of 5.
5. Update `UserStats.totalXp` and `UserStats.lastActivityDate`.
6. Run `computeStreak` to update `currentStreak` and `longestStreak`. The function classifies the outcome as `SAME_DAY`, `CONTINUED`, `RESET` or `NEW`; only the `CONTINUED` and `NEW` outcomes can lift the longest streak.
7. Insert an `ExerciseAttempt` row.
8. Check whether the user has now completed every exercise in the unit; if so award 100 diamonds with the reason `UNIT_COMPLETION`.
9. If `currentStreak` is a positive multiple of 7 award 150 diamonds with the reason `STREAK_7`.
10. Forward the XP delta to the leaderboard service so the active group's sorted set is updated.

The full method returns a `CompleteExerciseResult` object containing every delta — XP gained, diamonds gained, hearts remaining, the streak outcome, and whether the unit was completed — so the client can animate them one after another instead of polling for state.

9.2 Reward constants

The numerical reward values are kept in a single file at [backend/src/gamification/rewards.constants.ts](#) so they can be tuned without crawling the codebase.

Constant	Value	Meaning
<code>EXERCISE_BASE_XP</code>	20	Awarded for any completed exercise.
<code>PERFECT_SCORE_BONUS_XP</code>	5	Added when <code>correctCount === totalCount</code> .
<code>UNIT_COMPLETION_DIAMONDS</code>	100	Awarded the first time every exercise in a unit has been completed.

STREAK_7_DIAMONDS	150	Awarded when currentStreak % 7 === 0.
MAX_HEARTS	5	Hard cap on the heart pool.
HEART_REGEN_INTERVAL_MS	1,800,000	Thirty minutes per regenerated heart.
HEART_REFILL_COST_DIAMONDS	10	Per heart, when the user spends diamonds to skip the wait.

9.3 Hearts

The heart logic lives in [backend/src/gamification/hearts.util.ts](#) and is intentionally pure — every function takes the previous state and returns the next state without touching the database or the clock directly. A separate adapter in the service supplies "now" and persists the result. Two functions matter.

`applyHeartRegen(stored, lastRefillAt, now)` returns the number of hearts the user actually has at this moment. If `stored` is already at the cap it returns the cap. Otherwise it computes how many full thirty-minute windows have elapsed since `lastRefillAt`, adds that to `stored`, clips to the cap, and returns both the new heart count and the timestamp of the next regeneration tick.

`loseHearts(state, delta)` deducts hearts and, importantly, starts the regeneration timer the first time a user drops below the cap. If the user is already below the cap and the timer is running, it leaves the timer alone.

This split means the same logic runs identically when the mobile client wants to display the next-heart countdown and when the server processes a completion request.

9.4 Streaks

The streak computation in [backend/src/gamification/streak.util.ts](#) is timezone-aware. It uses the user's stored timezone (default "UTC") to derive a YYYY-MM-DD key for both the previous activity date and "now". The decision tree is small.

Comparison	Outcome	Effect on streak
<code>lastActivity == null</code>	NEW	<code>currentStreak = 1</code>
<code>lastKey == todayKey</code>	SAME_DAY	unchanged

lastKey == yesterdayKey	CONTINUED	currentStreak += 1
otherwise	RESET	currentStreak = 1

The cron job in `JobsModule` runs once a day at one minute past midnight UTC. It walks the `User` table and, for any user whose `lastActivityDate` is older than yesterday and whose `currentStreak` is positive, sets `currentStreak` back to zero. This ensures that a user who simply does not open the app sees the streak break the next time they look at it, instead of seeing a stale value.

10. Leaderboard Subsystem

The leaderboard is the social half of the gamification loop. The design was constrained by two requirements: a user must always be in some group (no empty leaderboard screen), and updates must be cheap because they fire on every completed exercise.

10.1 The group state machine

A `LeaderboardGroup` row carries one of three states.

- **WAITING** — fewer than twelve participants. The group is open for new joins and seasonal XP is *not yet* counted.
- **ACTIVE** — exactly twelve participants. The ten-day timer is running. Every XP delta from a completed exercise is added to the participant's `seasonXp`.
- **CLOSED** — the timer expired. The group's ranking is frozen and used to compute promotion or demotion before the participants are placed into new groups.

When a user posts to `POST /api/leaderboard/join` (or implicitly through their first XP-earning event of the season), the service searches for a `WAITING` group at the user's `currentLeagueIndex`. If one exists they are added; if none exists a new group is created. When the join brings `filledCount` to 12 the state flips to `ACTIVE`, `startedAt` is set to now and `endsAt` is set ten days out.

10.2 Promotion and demotion

When a group's timer expires its participants are sorted by `seasonXp` descending, with `lastXpAt` ascending used as the tiebreaker (an early completer beats a late one at the same XP). The top five promote one tier, the middle four hold, and the bottom three demote. Tier 1 is sticky on the floor — demotion from Tier 1 has no effect — and Tier 30

is sticky on the ceiling. Every participant's currentLeagueIndex on UserStats is updated accordingly and the group itself is marked CLOSED.

10.3 Use of Redis

The relational store is sufficient for correctness, but it is not the right structure for the read pattern. Showing the leaderboard means producing a sorted list of twelve users with $O(1)$ lookup of the requesting user's own rank. Doing that with SQL means a sort over the participants table on every request, which is wasteful. Redis sorted sets are the natural fit: a key per group (leaderboard: {groupId}) holds twelve members keyed by userId with seasonXp as the score; updates use ZINCRBY and reads use ZREVRANGE WITHSCORES and ZREVRANK. Both operations are $O(\log N)$.

The Redis client in [backend/src/redis/redis.module.ts](#) is wrapped in a factory that returns null when REDIS_URL is absent. The leaderboard service checks for null and falls back to the relational query, so the application still works in a development environment that has no Redis at all. In production the REDIS_URL points at an Upstash instance, which provides a managed serverless Redis with TLS.

11. Mobile Application

11.1 Stack

The mobile client is built with Expo SDK 51, React Native 0.74.5 and expo-router 3.5.23. File-system routing is used throughout, which means the navigation graph is the directory tree under mobile/app/. Persistent local state is stored through @react-native-async-storage/async-storage. The dark theme, including the green accent borrowed from Duolingo, is defined in [mobile/src/theme/colors.ts](#) and applied through a thin typography helper in the same folder.

11.2 Navigation graph

Route file	Purpose
mobile/app/_layout.tsx	Wraps the navigator in AuthProvider, ProgressProvider and CourseProvider.
mobile/app/index.tsx	Boot router: routes to /auth, /onboarding or /(tabs)/home depending on state.

mobile/app/auth.tsx	Combined login/registration form.
mobile/app/onboarding.tsx	Either-exam-or-subject enrollment flow.
mobile/app/(tabs)/home.tsx	Per-subject feed of units and exercises.
mobile/app/(tabs)/leaderboard.tsx	Current group, ranking, countdown.
mobile/app/(tabs)/profile.tsx	User identity, sign out, settings.
mobile/app/exercise/[id].tsx	Question-by-question runner, one submit at the end.
mobile/app/streak.tsx	Streak detail page.
mobile/app/shop.tsx	Heart refill and (placeholder) cosmetics.

11.3 State management

State is managed through three React contexts, each defined under [mobile/src/context/](#).

- **AuthContext** holds the current user, the JWT, the enrollments and a ready flag the boot router waits on.
- **ProgressContext** holds the gamified state (totalXp, diamonds, hearts, currentStreak, currentLeagueIndex, nextHeartAt) and exposes refreshStats and refillHeart.
- **CourseContext** caches the exam/subject/unit/exercise lists so navigation between screens does not re-hit the network.

The decision to use vanilla React Context rather than Redux or Zustand was deliberate. The state shape is small, the producers and consumers are localised, and the React DevTools profiler shows no rerender storms. Pulling in a separate state library would have added build size without buying anything.

11.4 The exercise runner and question renderers

The exercise runner walks the questions one at a time. For each question it consults a dispatcher component, [QuestionRenderer.tsx](#), which routes to one of five concrete renderers based on question.type.

Component	Type tag	Behaviour
-----------	----------	-----------

MCQ.tsx	MCQ	Renders the options, locks the choice on tap, marks correct/incorrect.
Match.tsx	MATCH	Renders two columns; the user taps left then right to form pairs.
Reorder.tsx	REORDER	Renders a draggable list of items.
Complete.tsx	COMPLETE	Renders a sentence with a blank and a single text input.
Translate.tsx	TRANSLATE	Renders source text and a free-text input for the translation.

Each renderer reports back through a single `onAnswered(correct: boolean)` callback. The runner aggregates these into a `boolean[]` and posts it to `POST /api/exercise/complete` at the end of the exercise. The server's response — XP gained, diamonds gained, hearts left, streak outcome, unit completion — drives the post-exercise summary screen.

11.5 The API client

The HTTP layer lives in `mobile/src/api/client.ts`. It is intentionally thin: a single `request<T>` function does the fetch, attaches `Authorization: Bearer <jwt>` if the token is present, throws on non-2xx with a typed error object, and returns `await res.json()`. Each public method (`api.login`, `api.completeExercise`, ...) is a one-line wrapper around `request`. The `EXPO_PUBLIC_API_URL` environment variable selects between local development and the production droplet URL.

12. Administrative Dashboard

The admin dashboard at `admin/` is a Next.js 14 application with TailwindCSS for styling, Radix UI for primitive components, and React Hook Form with Zod for form management. It is intended to be operated by the project owner only and is gated by an `ADMIN_API_TOKEN` rather than a multi-user role system.

The dashboard exposes one CRUD screen per content table — subjects, exams, units, exercises and questions — together with a small overview screen that shows row counts. Each list view is paginated, each create/edit flow is driven by a Zod schema that

mirrors the corresponding NestJS DTO, and all writes go through `lib/admin-api.ts`, which adds the admin token header on the server side so it never reaches the browser.

The privacy policy required for the Play Store submission is also served from this app, at `/privacy`. Hosting it inside the same deployment removes the need for a separate static site.

13. Content Authoring and Seed Pipeline

The seed pipeline is what turns a blank database into a usable product. It lives under `backend/prisma/` and consists of three layers.

The bottom layer is the per-subject question bank. Each file in `backend/prisma/seed-data/` (for example `cpp.ts`, `dsa.ts`, `python.ts`) exports a typed list of question records. Each record carries the question text, the answer payload and an explanation. These files are deliberately authored by hand so the wording quality is controllable.

The middle layer is `seed-bank.ts`, which assembles the per-subject banks into a `(subjectName, level) → Unit[]` map. It owns the slug generator and is the only file that knows how to spell each subject.

The top layer is `seed.ts`, which runs the pipeline. It performs the operations in dependency order: clear existing rows, insert subjects across three levels, insert units, insert exercises, insert questions, and finally link subjects to the appropriate exams. After the seed runs the database carries:

- 5 exams (`jee-main-2026`, `neet-2026`, `nimcet-2026`, `cuets-2026`, `upsc-cse-2026`)
- 24 subject rows (8 distinct subjects across 3 levels each, minus a few that only exist at certain levels)
- 124 units
- 465 exercises
- 4,650 questions

The full pipeline runs in roughly seven seconds against a local Postgres and is invoked through `npm run seed`. The same script is invoked automatically by `backend/deploy.sh` when the deploy script detects an empty Subject table on a fresh droplet.

14. Deployment and DevOps

14.1 Container layout

The backend is containerised through a multi-stage Dockerfile at [backend/Dockerfile](#). The build stage installs dependencies, runs `prisma generate`, compiles TypeScript with `nest build`, and prunes development dependencies. The runtime stage starts from a fresh `node:20-alpine`, copies in the build output, the pruned `node_modules`, and the Prisma schema, installs `ts-node` and `typescript` globally so the seed script can run inside the container, and exposes port 3000. The default command is `npx prisma migrate deploy && node dist/src/main.js`, which means every container start automatically applies any pending migrations before booting the API.

The compose file at [backend/docker-compose.yml](#) wires the API container to a `postgres:16-alpine` container with a named volume for persistent storage, and binds the API to `127.0.0.1:3001` on the host so that an existing reverse proxy can route to it. Redis is *not* part of compose; it is provided by Upstash and reached over TLS through `REDIS_URL`.

14.2 The deploy script

[backend/deploy.sh](#) is a small Bash script that performs an idempotent deploy: it pulls the Postgres image, builds the API image, starts the database, waits for `pg_isready`, starts the API (which runs migrations on its own), and runs the seed if and only if the Subject table is empty. Subsequent deploys after a `git pull` are a single command:

```
cd ~/Locus/backend && git pull && ./deploy.sh
```

14.3 Production environment

The production environment is a single 2 GB DigitalOcean droplet running Ubuntu 24.04 in the BLR1 region. The droplet hosts the API container, the Postgres container, and a small set of unrelated services. Redis is provided by Upstash on a free-tier instance located in the same region. The mobile client reaches the API through a public IP on port 3001; the longer-term plan documented in section 19 is to put the API behind a domain with TLS through the host's existing reverse proxy.

The signed Android App Bundle is built locally with the standard Gradle pipeline (`./gradlew bundleRelease`), signed with a 10,000-day RSA-2048 keystore generated for the project, and uploaded to the Google Play Console on the internal testing track. Two patches were required to make the bundle pass Play's pre-launch checks: one to

handle a null array in expo-modules-core's permissions service when compileSdk is raised to 35, and one to replace a Kotlin removeLast() call in react-native-screens with removeAt(list.lastIndex) to avoid a documented crash on Android 15.

15. Security Considerations

Security on a small platform is mostly about getting the basics right.

- **Password storage.** Passwords are bcrypt-hashed with a cost factor of 10 and are never returned by any endpoint.
- **Authentication.** All authenticated endpoints sit behind JwtAuthGuard. The JWT secret is loaded from the environment and is rotated separately from application code.
- **Authorisation.** Admin endpoints require a static admin token in the x-admin-token header. This is intentionally the simplest viable scheme; multi-user RBAC was not in scope.
- **Validation.** Every controller uses class-validator DTOs with whitelist: true and forbidNonWhitelisted: false, which strips unknown fields before they reach the service. This closes a class of mass-assignment problems.
- **Database access.** Prisma is the only path into the database, which means every query is parameterised. There is no string-concatenated SQL anywhere in the codebase.
- **Transport.** The mobile client talks to the API over HTTP at the time of writing. The next iteration will route traffic through a TLS-terminating reverse proxy and switch the client to HTTPS-only.
- **Secret hygiene.** Production secrets — the JWT signing key, the Postgres password, the admin token, the Upstash URL — are present only in the droplet's .env file, which is excluded from version control through .gitignore. A committed .env.example documents the required keys without the values.

16. Testing and Validation

The testing approach was pragmatic and emphasised end-to-end behaviour over unit coverage.

The backend's pure utility modules — streak.util.ts, hearts.util.ts and the reward calculation in gamification.service.ts — were exercised through small

driver scripts that invoked them with hand-crafted inputs and printed the results, which were then visually compared with the gamification specification.

The HTTP surface was validated with `curl` against a locally-running container and against the production droplet. A scripted scenario walks through the lifecycle of a new user: `POST /auth/register`, `POST /me/enrollments/exam`, `GET /courses/exams`, `GET /courses/exercises/:id`, `POST /exercise/complete`, `GET /stats`. The expected XP, hearts and streak deltas were checked at each step. Negative cases — completing an exercise with zero hearts, posting an answer array of the wrong length, requesting a missing exercise — were also exercised.

The mobile client was validated on a physical Android device throughout development. The pre-launch report from Google Play's automated robot testing on the AAB caught the two compatibility issues mentioned in section 14 and confirmed that the production build starts cleanly on Android 14 and 15 across multiple device profiles.

17. Results

The platform is live. As of submission:

- The signed AAB is available on the Google Play internal testing track and installs on a stock Pixel emulator without warning.
- The API is reachable at <http://144.126.255.209:3001/api>, returning live JSON for every documented endpoint.
- The seed pipeline produces 24 subjects, 124 units, 465 exercises and 4,650 questions in roughly seven seconds.
- A new user can register, enroll, complete an exercise, lose a heart, gain XP, and be placed into a leaderboard group — the whole loop — in under a minute on a 2 GB droplet.
- The total codebase is approximately 4,800 lines of TypeScript spread across the four sub-projects, plus the Prisma schema and seed data.

The platform meets every primary objective listed in section 4.1.

18. Limitations

Several limitations remain.

- **Transport security.** The API is served over plain HTTP. A reverse proxy with Let's Encrypt termination is the next priority.
- **No real-time push.** Streak warnings and league-end notifications require the user to open the app. Adding Expo push notifications would close that loop.
- **Single-region infrastructure.** The droplet and the Upstash instance are both in BLR1. The mobile client's latency outside India is therefore higher than necessary.
- **Manual question authoring.** While the seed pipeline is deterministic, the question text itself is hand-written. A larger corpus will require either a content team or a question-generation tool.
- **No analytics.** The platform records ExerciseAttempt rows but does not yet aggregate them into a learner-facing progress dashboard or an operator-facing engagement report.

19. Future Work

The roadmap beyond the capstone milestone is grouped into three themes.

Reach. Buy a domain, put Caddy in front of the API for automatic TLS, add the Play Store production track, build for iOS through EAS.

Depth. Expand the corpus to roughly 50,000 questions across the existing five exams; add a sixth exam (CAT) that requires a verbal-reasoning question type; introduce an adaptive selector that picks the next exercise based on the learner's recent error pattern instead of strict ordering.

Engagement. Add Expo push notifications for streak warnings; add a "streak freeze" item priced in diamonds; introduce friend-based leagues alongside the anonymous twelve-person ones.

20. Conclusion

Locus demonstrates that a single developer, working with mainstream open-source tooling, can ship a vertically integrated learning product — schema, server, mobile app and content pipeline — to a production environment within the timeline of a capstone project. The technical decisions made along the way (Prisma over hand-written SQL, JWT over session cookies, Expo over bare React Native, Docker Compose over a managed Kubernetes cluster, a flat JSON content payload over a per-question-type

table) all traded a small amount of sophistication for a large amount of reproducibility, and that trade has paid off in deployment speed.

The product itself is small, but it is honest: the streak counter is real, the leaderboard sorts on real seasonal XP, the heart timer ticks down at the same rate on the server and the client, and the question bank is large enough that an early user can practise meaningfully without immediately exhausting it. The mechanics that make Duolingo retentive in language learning appear to translate cleanly to entrance-exam preparation, which was the central hypothesis behind the project.

The remaining work — TLS, push notifications, a wider corpus, an adaptive selector — is well-scoped and incremental. The architecture documented in this report supports each of those additions without requiring redesign.

21. References

1. NestJS framework documentation. <https://docs.nestjs.com> (accessed April 2026).
2. Prisma documentation. <https://www.prisma.io/docs> (accessed April 2026).
3. PostgreSQL 16 documentation. <https://www.postgresql.org/docs/16/> (accessed April 2026).
4. Redis sorted set commands. <https://redis.io/docs/data-types/sorted-sets/> (accessed April 2026).
5. Expo SDK 51 release notes. <https://docs.expo.dev/versions/v51.0.0> (accessed April 2026).
6. React Native 0.74 documentation. <https://reactnative.dev/docs/0.74/getting-started> (accessed April 2026).
7. Next.js 14 App Router documentation. <https://nextjs.org/docs/app> (accessed April 2026).
8. Google Play target API level requirements. <https://developer.android.com/google/play/requirements/target-sdk> (accessed April 2026).
9. JSON Web Token specification (RFC 7519). <https://datatracker.ietf.org/doc/html/rfc7519>.
10. bcrypt password hashing. Provos, N. and Mazières, D. "A future-adaptable password scheme." USENIX Annual Technical Conference, 1999.
11. DigitalOcean Droplets documentation. <https://docs.digitalocean.com/products/droplets/> (accessed April 2026).

12. Upstash Redis documentation. <https://docs.upstash.com/redis> (accessed April 2026).

Appendix A — Sample API Requests

The following requests illustrate the most common interactions. The base URL is <http://144.126.255.209:3001/api> in production.

Register a user

POST /api/auth/register
Content-Type: application/json

```
{  
  "email": "demo@example.com",  
  "password": "demo-1234",  
  "displayName": "Demo",  
  "timezone": "Asia/Kolkata"  
}
```

Response:

```
{  
  "user": { "id": "cuid...", "email": "demo@example.com",  
    "displayName": "Demo" },  
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."  
}
```

Enroll in an exam

POST /api/me/enrollments/exam
Authorization: Bearer <jwt>
Content-Type: application/json

```
{ "examId": "cuid...", "examDate": "2026-09-01" }
```

Complete an exercise

POST /api/exercise/complete

Authorization: Bearer <jwt>

Content-Type: application/json

```
{ "exerciseId": "cuid...", "results":  
[true,true,false,true,true,true,true,true,true,true] }
```

Response (truncated):

```
{  
  "scorePct": 90,  
  "isPerfect": false,  
  "xpAwarded": 20,  
  "diamondsAwarded": 0,  
  "heartsRemaining": 4,  
  "streakOutcome": "CONTINUED",  
  "currentStreak": 3,  
  "unitCompleted": false  
}
```

Appendix B — Question Content Schemas

Question.content is stored as a JSON string. The runtime schema for each type is below.

MCQ

```
{ "options": ["A", "B", "C", "D"], "correct_index": 0 }
```

MATCH

```
{ "left": ["Apple", "Cat"], "right": ["Fruit", "Animal"], "map":  
{ "Apple": "Fruit", "Cat": "Animal" } }
```

REORDER

```
{ "items": ["Step 1", "Step 2", "Step 3"], "order": [0, 1, 2] }
```

COMPLETE

```
{ "blank_text": "The capital of India is __", "answer": "Delhi" }
```

TRANSLATE

```
{ "source_text": "Hello", "source_lang": "en", "target_lang": "hi",  
  "answer": "नमस्ते" }
```

Appendix C — File and Module Inventory

Approximate line counts (TypeScript only, excluding generated code and node_modules).

Sub-project	Files of interest	LOC
Backend	src/ (10 modules), prisma/schema.prisma, prisma/seed*.ts	~2,000
Mobile	app/, src/ (api, components, context, theme)	~1,400
Admin	app/, src/components, src/lib	~1,000
Course data	Courses/, prisma/seed-data/	~360
Total		~4,760

Key files referenced throughout the report:

- [backend/src/main.ts](#) — bootstrap
- [backend/src/app.module.ts](#) — module composition
- [backend/src/auth/auth.service.ts](#) — registration and login
- [backend/src/courses/courses.service.ts](#) — read-only content endpoints
- [backend/src/gamification/gamification.service.ts](#) — completion path
- [backend/src/gamification/streak.util.ts](#) — streak computation
- [backend/src/gamification/hearts.util.ts](#) — heart regen
- [backend/src/gamification/rewards.constants.ts](#) — reward values

- [backend/src/leaderboard/leaderboard.service.ts](#) — league mechanics
- [backend/prisma/schema.prisma](#) — full data model
- [backend/prisma/seed.ts](#) — seed pipeline
- [backend/Dockerfile](#) — production image
- [backend/docker-compose.yml](#) — orchestration
- [backend/deploy.sh](#) — droplet deploy script
- [mobile/app/_layout.tsx](#) — root layout
- [mobile/src/api/client.ts](#) — HTTP client
- [mobile/src/components/questions/QuestionRenderer.tsx](#) — type dispatcher
- [admin/src/app/privacy/page.tsx](#) — privacy policy

End of report.